CANDY
DIALOGUE ENGINE

Lexicon

1.1.0

The Lexicon automatically highlights important names, terms, and keywords inside your dialogue text by changing their formatting. Lexicon entries can also display tooltips when hovered, and can be made clickable with fully customizable effects.

Uses

The Lexicon feature can be used for drawing attention to certain words or information, or just for aesthetic purposes.

It can also be used to provide useful information to the player through tooltips, or by opening a database entry when clicked.

Furthermore, keywords can be individually programmed to do anything when clicked. You could potentially use this for game control or dialogue flow, such as enabling the player to click words to perform actions in the game or to navigate dialogues.

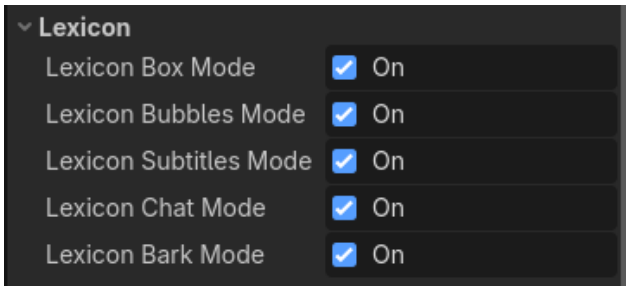
BBCode

Lexicon keywords are formatted with BBCode tags, so any formatting is limited by Godot itself.

Make sure the node you use for text display supports BBCode (e.g. RichTextLabel) and that the 'BBCode' property is enabled in the node.

Enabling Lexicon

The Lexicon feature will only work for specific dialogue modes if the **lexicon variables** in **Candy_Engine.gd** are set to 'true' for each mode:



If you use the Lexicon feature, you can selectively set these variables to 'false' at runtime to momentarily disable the Lexicon, for example during subtitled cutscenes.

Lexicon Dictionary

Each keyword has its own custom formatting settings, defined inside the **lexicon** dictionary, located in **Candy_Database.gd**

Variants

If you haven't read the **Variants & Translations** guide yet, variants are alternate texts for spoken lines. They're used for translating dialogues to other languages, or for alternate variations of a line in a same language.

The **lexicon** dictionary is sorted by variants first. This way, each variant or language can have its own keywords.

If you want a variant to be an exact match, just add a key that matches the variant exactly. For example, entries under **"French"** will only trigger if the selected variant is named "French" *exactly*.

If you want looser matching, add an asterisk (*) as a suffix to the key. For example, keywords under **"French*"** will trigger if the selected variant's name *begins with* "French". Therefore, "French" and "French_#1" will both trigger keywords under **"French*"**.

If you want some entries to be used by all variants, place them under a key named **"*"**:

```
var lexicon = {
	"*": {
		"Alice": {
			"Display": "",
			"BBCode": ["[b][color=green]", "[/color][/b]"],
			"Tooltip": "Alice is a student.",
			"Clickable": false,
			"Click_Data": [],
		},
	},
}
```

If a keyword is found under multiple variants, the closest/most specific match will be used. For example, if the keyword 'Alice' is found under **"*"**, **"French*"** and **"French_1*"**, the variant "French_1" will use the 'Alice' entry in **"French_1*"** because that's more specific than the others.

Exact matches will always be prioritized. For example, if 'Alice' is found under **"French_1"** and **"French_1*"**, the variant "French_1" will use the 'Alice' entry under **"French_1"**.

Remember that the original text of a line is always saved as a variant named "Default", no matter which language you write in.

If you want some Lexicon entries to only be triggered when the original text is displayed, place those entries under either **"Default"** or **"Default*"**.

With variants now explained, let's look at how to configure Lexicon keywords.

Settings

```
var lexicon = {
        "*": {
                "Alice": {
                        "Display": "",
                        "BBCode": ["[b][color=green]", "[/color][/b]"],
                        "Tooltip": "Alice is a student.",
                        "Clickable": false,
                        "Click_Data": [],
                },
        },
}
```

Each top key (e.g. "Alice") represents a Lexicon keyword to format, while the sub-keys determine the formatting that is applied:

Never use the following symbols in keywords:

\ . ^ $ | ( ) [ ] { } * + ?

Also, don't use the following Candy DE symbols:

text_var_symbol_start

text_var_symbol_end

substitution_symbol

separator_symbol

Display

The **"Display"** key is how the keyword should be written. For example, you could use this to replace the 'Alice' keyword with "Alice Turner" wherever it appears in spoken text.

Alternately, you can use this in case some words can have different meanings and shouldn't always have Lexicon formatting applied. For example, 'June' can be both a first name and a month. If you want to format it when it refers to the character named June, but not when it refers to the month, you could add a prefix to the keyword, such as **"_June"**, and enter "June" in the **"Display"** key:

```
"Default*":{
        "_June": {
                "Display": "June",
                "BBCode": ["[b][color=blue]", "[/color][/b]"],
                "Tooltip": "Alice is a student.",
                "Clickable": false,
                "Click_Data": [],
        },
},
```

In spoken lines, you would write '_June' wherever you refer to the character, and it would automatically be formatted with BBCode and changed to "June". Meanwhile, if you write 'June' in spoken lines, it won't be formatted because that isn't a keyword in the Lexicon dictionary.

If the **"Display"** key is missing or if it's empty (""), the keyword will be displayed as-is (e.g. "_June", not "June").

BBCode

The **"BBCode"** key is an array of two strings and represents the BBCode formatting you want to apply to the keyword.

Feel free to add any BBCode tags you want. Opening tags (e.g. [b], [i], etc.) go in the first string, while closing tags (e.g. [/b], [/i], etc.) go in the second string.

You'll have to exert common sense, as some tags may not be practical (e.g. text alignment tags).

Never use the [tooltip] and [url] tags, as those are reserved by the Lexicon feature.

Tooltip

The "Tooltip" option currently doesn't work with the default speech bubbles. We're going to look into other bubble designs to resolve this.

It should be an easy fix for 2D bubbles, but for 3D bubbles, options are limited by Godot.

The **"Tooltip"** key contains the text you want the tooltip to display. When the **"Tooltip"** key is not empty (""), the engine adds the [tooltip][/tooltip] BBCode tags around the keyword.

These tags make the keyword display a tooltip when hovered with the mouse. Candy DE will use the **"Tooltip"** value as the tooltip text to display.

Clickable

The "Clickable" option currently doesn't work with the default speech bubbles. We're going to look into other bubble designs to resolve this.

It should be an easy fix for 2D bubbles, but for 3D bubbles, options are limited by Godot.

The **"Clickable"** key can be set to true if you want to make a keyword clickable with the mouse. If so, the engine will add the [url][/url] BBCode tags around the word.

In Godot, the [url] tag makes a word clickable, and when clicked, the RichTextLabel emits the **meta_clicked()** signal.

This is where things get a bit technical, and you may require intermediate-level skills in GDScript. Don't feel intimidated: you only need to write one (very) simple function. What follows is just an explanation of the arguments that function receives, and how clicking keywords triggers it.

When keywords are clicked, the **meta_clicked()** signal has no effect by default: it's just a signal being emitted into the void.

Thankfully, the included default UI elements (like the Default dialogue box) connect the signal to the **_on_lexicon_clicked()** function, which is located in the **Candy_Functions.gd** script. If you're a beginner:

click → emit signal → call function

In Candy DE, the **meta_clicked()** signal carries one argument: a dictionary, containing the clicked keyword ('keyword'), the keyword's parent variant in the Lexicon dictionary ('keyword_variant'), and the keyword's **"Click_Data"** value ('keyword_click_data'):

```
var meta_dict = {
        "keyword": word,
        "keyword_click_data": keyword_click_data,
        "keyword_variant": variant_key,
}
```

The **_on_lexicon_clicked()** function receives three arguments: the signal's dictionary argument ('data'), the node that runs the dialogue engine instance ('caller') and the UI element that displayed the text featuring the keyword ('ui'):

```
func _on_lexicon_clicked(data, caller, ui):
        var keyword = data["keyword"]
        var keyword_data = data["keyword_click_data"]
        var keyword_variant = data["keyword_variant"]
        pass
```

Note that 'ui' is the root node of the UI element, not the RichTextLabel that displayed the text. 'keyword' is the name of the keyword in the Lexicon dictionary, not how it is displayed on screen (if you use the **"Display"** key).

The rest is up to you: you'll need to write the logic for **_on_lexicon_clicked()** depending on what you want to happen when a keyword is clicked. The arguments passed to the function should serve any use you may have.

Once you understand the data that is passed to the _on_lexicon_clicked() function, this is actually very easy: you'll probably just write code to make a database window visible, then use **keyword** and **keyword_variant** to have it display the correct information.

Note that you may want to pause dialogue when a keyword is clicked. See the **Suspend Dialogue** guide for details.

If you want to pass more arguments to the **meta_clicked()** signal, you can modify the **apply_lexicon_tags()** function in **Candy_Engine.gd** to add more keys to the **meta_dict** dictionary.

If you create your own UI elements for displaying spoken text, be sure to connect the **meta_clicked()** signal to the **_on_lexicon_clicked()** function in your UI element's **_ready()**. You can just copy the code from the provided Default dialogue box (or from other Default UI elements), and use it as a template.

Additionally, you'll probably want to disable meta_underlined and hint_underlined in RichTextLabels that display spoken text, under Markup settings: